

Wallet Ledger Service

A ledger-based wallet service built with raw Node.js HTTP server, Prisma ORM, and PostgreSQL advisory locks for concurrency-safe transactions.

Overview

This monorepo implements a wallet service with four REST endpoints for managing user balances and purchases. The core design principle is **ledger-based accounting**: balances are never stored directly but computed from the sum of all transaction entries. This approach provides an immutable audit trail and eliminates balance drift from bugs or race conditions.

Architecture

```
wallet-ledger-demo/
├── apps/
│   └── wallet/           # HTTP API server (Node.js)
├── packages/
│   ├── db/              # Prisma schema + client generation
│   ├── seed/            # Catalog data seeder (dummyjson.com)
│   └── types/            # Shared TypeScript types
└── tooling/             # ESLint, Prettier, TypeScript configs
```

Running Locally

Prerequisites

- Node.js 24+
- pnpm 9+
- PostgreSQL database (or Prisma Postgres URL)
- jq (for environment setup)

1. Environment Setup

A password-protected `env-scaffold.zip` is provided separately. Unzip it at the repository root:

```
unzip env-scaffold.zip
```

This creates `env-scaffold.json` with the required database credentials.

Run the environment seeder to distribute values to all workspace packages:

```
pnpm env:seed
```

This generates `.env` files in:

- `apps/wallet/.env` - API server configuration
- `packages/db/.env` - Prisma client
- `packages/seed/.env` - Data seeder
- `./env` - Root (turborepo)

2. Install Dependencies

```
pnpm install
```

3. Run Migrations

Generate Prisma client and apply migrations:

```
# From repository root
pnpm --filter @wallet-ledger/db db:generate
pnpm --filter @wallet-ledger/db db:migrate
```

Or from the wallet app:

```
cd apps/wallet
pnpm db:migrate
```

4. Build Packages

```
pnpm build:db && pnpm build:types && pnpm build:seed && pnpm build:wallet
```

5. Start the Server

```
# Development (hot reload)
pnpm run:wallet

# Or directly
cd apps/wallet && pnpm dev
```

The server starts on <http://localhost:3000> (configurable via `PORT` env var).

API Endpoints

All endpoints require the `x-user-id` header with a valid UUID v1-5.

GET /api/items

Returns the catalog of 50 available items (prices in cents).

```
curl http://localhost:3000/api/items \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000"
```

Response:

```
[
  {"id": "ab1efe30-4fb5-4431-89cd-907c4b028234", "name": "Durango SXT RWD",
  "price": 3699999},
  {"id": "674236b1-14fd-4ac3-9cc0-450ef47ddc56", "name": "Charger SXT RWD",
  "price": 3299999},
  ...
]
```

GET /api/balance

Returns the user's current balance (computed from ledger entries).

```
curl http://localhost:3000/api/balance \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000"
```

Response:

```
{"balance": 0}
```

POST /api/credits

Adds credits to the user's wallet. Returns the new balance.

```
curl -X POST http://localhost:3000/api/credits \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000" \
  -H "Content-Type: application/json" \
  -d '{"amount": 5000000}'
```

Response:

```
{"balance": 5000000}
```

POST /api/purchases

Purchases an item, deducting from the user's balance.

```
curl -X POST http://localhost:3000/api/purchases \
  -H "x-user-id: 550e8400-e29b-41d4-a716-446655440000" \
  -H "Content-Type: application/json" \
  -d '{"itemId": "ab1efe30-4fb5-4431-89cd-907c4b028234}"'
```

Response: 204 No Content

Concurrency Safety: PostgreSQL Advisory Locks

The purchase endpoint must prevent overdrafts when concurrent requests arrive for the same user. Consider two requests attempting to purchase a \$1000 item when the user has exactly \$1000:

Without protection, both requests could:

1. Read balance: \$1000
2. Check: \$1000 >= \$1000 (pass)
3. Create debit: -\$1000
4. Result: Balance is -\$1000 (overdraft)

Solution: Transaction-Scoped Advisory Locks

PostgreSQL advisory locks provide application-level locking without row contention. The implementation:

```
async purchaseItem(userId: string, item: CatalogItem): Promise<PurchaseResult> {
  return await prisma.$transaction(async (tx) => {
    // 1. Acquire per-user advisory lock
    const lockKey = uuidToLockKey(userId);
    await tx.$executeRawUnsafe(`SELECT pg_advisory_xact_lock($1)`, lockKey);

    // 2. Calculate balance WITHIN transaction (after lock acquired)
    const balance = await calculateBalance(tx, userId);

    // 3. Check sufficient funds
    if (balance < item.price) {
      return { success: false, insufficientFunds: true };
    }

    // 4. Create debit entry with price snapshot
    await tx.ledgerEntry.create({
      data: {
        userId,
        entryType: "DEBIT",
        amount: -BigInt(item.price),
        debitDetail: {
          create: {
            itemId: item.id,
            itemName: item.name,
            amount: BigInt(item.price) // Price at purchase time
          }
        }
      }
    });

    return { success: true };
    // Lock auto-released on commit
  });
}
```

Lock Key Generation

The user's UUID is converted to a 64-bit lock key by XORing the high and low halves:

```
uuidToLockKey(uuid: string): bigint {
  const hex = uuid.replace(/-/g, "");
  const high = BigInt("0x" + hex.slice(0, 16));
  const low = BigInt("0x" + hex.slice(16));
  return high ^ low;
}
```

Why Advisory Locks?

Alternative	Problem
SELECT FOR UPDATE	Requires existing row; doesn't work for new users
Optimistic locking	Retry storms under high contention
Serializable isolation	Blocks all users, not just concurrent same-user requests
Application mutex	Doesn't scale across server instances

Advisory locks:

- Work for new users (no existing rows needed)

- Only block concurrent requests for the **same user**
- Auto-release on transaction commit/rollback
- Scale across server instances (lock is database-level)

Database Indexes

The schema includes indexes optimized for the wallet service's access patterns:

LedgerEntry Indexes

Index	Columns	Purpose
Primary	<code>id</code>	Row identification
<code>@@index([userId])</code>	<code>userId</code>	Balance calculation, user history
<code>@@index([userId, entryType])</code>	<code>userId, entryType</code>	Filter credits vs debits per user
<code>@@index([userId, status])</code>	<code>userId, status</code>	Filter COMPLETED entries for balance
<code>@@index([userId, createdAt(sort: Desc)])</code>	<code>userId, createdAt DESC</code>	Recent transactions per user
<code>@@index([entryType, createdAt(sort: Desc)])</code>	<code>entryType, createdAt DESC</code>	Admin: recent credits/debits system-wide
<code>@@index([status, createdAt])</code>	<code>status, createdAt</code>	Admin: failed/pending transaction monitoring

Balance Calculation Query:

```
SELECT SUM(amount) FROM "LedgerEntry"
WHERE "userId" = $1 AND "status" = 'COMPLETED'
```

Uses the `[userId, status]` composite index for optimal performance.

DebitDetail Indexes

Index	Columns	Purpose
Primary	<code>ledgerEntryId</code>	1:1 relation to LedgerEntry
<code>@@index([itemId])</code>	<code>itemId</code>	Item purchase history
<code>@@index([itemId, createdAt(sort: Desc)])</code>	<code>itemId, createdAt DESC</code>	Recent purchases of specific item

IdempotencyRecord Indexes

Index	Columns	Purpose
Primary	<code>id</code>	Row identification
<code>@@unique([key, userId])</code>	<code>key, userId</code>	Fast idempotency lookup
<code>@@unique([ledgerEntryId])</code>	<code>ledgerEntryId</code>	1:1 relation to LedgerEntry
<code>@@index([expiresAt])</code>	<code>expiresAt</code>	Cleanup job for expired records
<code>@@index([endpoint])</code>	<code>endpoint</code>	Filter by API endpoint
<code>@@index([statusCode])</code>	<code>statusCode</code>	Filter by response status

Index	Columns	Purpose
<code>@@index([key])</code>	key	Lookup by key alone
<code>@@index([userId])</code>	userId	User's idempotency records

Index Tradeoffs

- Storage overhead:** Each index consumes disk space proportional to the indexed columns. The `LedgerEntry` table has 6 indexes, which is reasonable for a write-moderate, read-heavy workload.
- Write amplification:** Every INSERT/UPDATE must update all affected indexes. For ledger entries (append-only), this is acceptable since we rarely update existing records.
- Covering indexes:** The composite indexes like `[userId, status]` are designed to "cover" common queries, avoiding heap lookups when only the indexed columns are needed.
- Missing indexes:** No index on `amount` since we always aggregate (SUM) rather than filter by amount. Adding one would slow writes without benefiting reads.

Idempotency Design Note

For production deployment, idempotency would be implemented using the existing `IdempotencyRecord` model in the database schema. Clients would include an `x-idempotency-key` header with mutating requests (POST `/api/credits` and POST `/api/purchases`). Before processing any request, the server checks for an existing record matching the composite key (`key`, `userId`). If found with a completed status, the cached `responseBody` and `statusCode` replay immediately without re-executing the operation. If not found, a new record is created with PENDING status before processing begins.

Upon successful completion, the record updates with the final response (status code and JSON body). For purchases specifically, the record links to the created `LedgerEntry` via `ledgerEntryId`, enabling correlation between idempotency records and actual transactions. The `endpoint` enum (CREDITS, PURCHASES, etc.) allows filtering records by operation type. Records include `retryCount` and `lastRetryAt` fields for monitoring duplicate request patterns.

Expired records are cleaned up based on the `expiresAt` timestamp, typically set to 24 hours after creation. This provides sufficient protection against duplicate submissions from network retries, client bugs, or user double-clicks while preventing unbounded storage growth.

Error Responses

Status	Condition
400	Missing or invalid <code>x-user-id</code> header
400	Invalid JSON body
400	<code>amount</code> not a positive integer
400	<code>itemId</code> not a valid UUID
404	Item not found in catalog
405	Method not allowed for endpoint
409	Insufficient balance for purchase
500	Internal server error

Testing

Run all tests:



```
cd apps/wallet
pnpm test
```

Run specific test suites:

```
pnpm test:validation # Input validation tests (23 tests)
pnpm test:api        # API endpoint tests (requires running server)
pnpm test:concurrency # Concurrent purchase stress tests
```

Concurrency Test

The concurrency test verifies advisory lock behavior by firing multiple simultaneous purchase requests:

```
# Terminal 1: Start server
pnpm run:wallet

# Terminal 2: Run concurrency tests
pnpm test:concurrency
```

The test:

1. Creates a user with exactly enough balance for N purchases
2. Fires N+1 concurrent purchase requests
3. Verifies exactly N succeed and 1 fails with 409 (insufficient funds)
4. Confirms final balance is 0 (no overdraft)

Tech Stack

- **Runtime:** Node.js 24+ (native test runner, ESM)
- **Database:** PostgreSQL via Prisma ORM 7.x
- **Logging:** Pino with pino-pretty for development
- **Build:** tsdown (Rolldown-based bundler)
- **Monorepo:** Turborepo + pnpm workspaces
- **TypeScript:** 5.9+ with strict mode

Scripts Reference

Script	Description
pnpm env:seed	Generate .env files from env-scaffold.json
pnpm build:db	Build database package
pnpm build:seed	Build seed package
pnpm build:wallet	Build wallet API
pnpm run:wallet	Start wallet API in dev mode
pnpm dev	Run all packages in parallel dev mode
pnpm clean:house	Deep clean and reinstall